

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

NAME:P. Varshitha

Roll no:2503A52398

Batch:14

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

INPUT:

```
class Stack:
```

```
    """Implementation of a Stack using Python list."""
```

```
    def __init__(self):
```

```
        self.items = []
```

```
    def push(self, item):
```

```
        """Add an item to the top of the stack."""
```

```
self.items.append(item)
```

```
def pop(self):
```

```
    """Remove and return the top item."""
```

```
    if self.is_empty():
```

```
        return "Stack is empty"
```

```
    return self.items.pop()
```

```
def peek(self):
```

```
    """Return the top item without removing it."""
```

```
    if self.is_empty():
```

```
        return "Stack is empty"
```

```
    return self.items[-1]
```

```
def is_empty(self):
```

```
    """Check whether the stack is empty."""
```

```
    return len(self.items) == 0
```

```
stack = Stack()
```

```
stack.push(10)
```

```
stack.push(20)
```

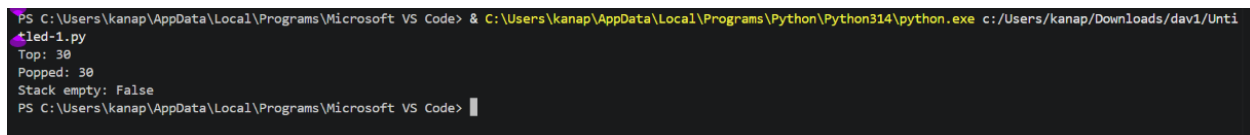
```
stack.push(30)
```

```
print("Top:", stack.peak())
```

```
print("Popped:", stack.pop())
```

```
print("Stack empty:", stack.is_empty())
```

OUTPUT:



```
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kanap\AppData\Local\Programs\Python\Python314\python.exe c:/Users/kanap/Downloads/dav1/Unti
led-1.py
Top: 30
Popped: 30
Stack empty: False
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

INPUT:

```
class Queue:
```

```
"""Implementation of a Queue using Python list."""
```

```
def __init__(self):
```

```
    self.items = []
```

```
def enqueue(self, item):
```

```
    """Add an item to the rear of the queue."""
```

```
    self.items.append(item)
```

```
def dequeue(self):
```

```
    """Remove and return the front item."""
```

```
    if self.size() == 0:
```

```
        return "Queue is empty"
```

```
    return self.items.pop(0)
```

```
def peek(self):
```

```
    """Return the front item without removing it."""
```

```
    if self.size() == 0:
```

```
        return "Queue is empty"
    return self.items[0]
```

```
def size(self):
    """Return the number of items in the queue."""
    return len(self.items)
```

```
queue = Queue()
queue.enqueue("A")
queue.enqueue("B")
queue.enqueue("C")
```

```
print("Front:", queue.peek())
print("Removed:", queue.dequeue())
print("Size:", queue.size())
```

OUTPUT:

```
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kanap\AppData\Local\Programs\Python\Python314\python.exe c:/Users/kanap/Downloads/dav1/Unti
led-1.py
Front: A
Removed: A
Size: 2
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> |
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

INPUT:

class Node:

"""Represents a node in a linked list."""

def __init__(self, data):

self.data = data

self.next = None

class LinkedList:

"""Implementation of a Singly Linked List."""

```
def __init__(self):
    self.head = None

def insert(self, data):
    """Insert a new node at the end."""
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    current = self.head

    while current.next:
        current = current.next

    current.next = new_node
```

```
def display(self):  
    """Display all elements of the linked list."""  
    current = self.head  
  
    while current:  
        print(current.data, end=" -> ")  
        current = current.next  
  
    print("None")
```

```
linked_list = LinkedList()
```

```
linked_list.insert(10)
```

```
linked_list.insert(20)
```

```
linked_list.insert(30)
```

```
linked_list.display()
```

OUTPUT:

```
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code\python.exe c:/Users/kanap/Downloads/dav1/Untitled-1.py
10 -> 20 -> 30 -> None
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> |
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

INPUT:

class Node:

"""Represents a node in a Binary Search Tree."""

def __init__(self, data):

self.data = data

self.left = None

self.right = None

```
class BST:
```

```
    """Binary Search Tree implementation."""
```

```
    def __init__(self):
```

```
        self.root = None
```

```
    def insert(self, root, data):
```

```
        """Recursively insert data into the BST."""
```

```
        if root is None:
```

```
            return Node(data)
```

```
        if data < root.data:
```

```
            root.left = self.insert(root.left, data)
```

```
        else:
```

```
            root.right = self.insert(root.right, data)
```

```
    return root
```

```
def inorder(self, root):  
    """Perform in-order traversal."""  
    if root:  
        self.inorder(root.left)  
        print(root.data, end=" ")  
        self.inorder(root.right)
```

```
bst = BST()
```

```
for value in [50, 30, 70, 20, 40, 60, 80]:  
    bst.root = bst.insert(bst.root, value)
```

```
print("In-order Traversal:")  
bst.inorder(bst.root)
```

OUTPUT:

```
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kanap\AppData\Local\Programs\Python\Python314\python.exe c:/Users/kanap/Downloads/dav1/Untitled-1.py
In-order Traversal:
20 30 40 50 60 70 80
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #5 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

INPUT:

```
import heapq
```

```
class PriorityQueue:
```

```
    """Priority Queue implementation using heapq."""
```

```
    def __init__(self):
```

```
        self.queue = []
```

```
    def enqueue(self, item, priority):
```

```
        """Add an item with a priority."""
```

```
heapq.heappush(self.queue, (priority, item))
```

```
def dequeue(self):
```

```
    """Remove and return the highest-priority item."""
```

```
    if not self.queue:
```

```
        return "Priority Queue is empty"
```

```
    priority, item = heapq.heappop(self.queue)
```

```
    return item
```

```
def display(self):
```

```
    """Display all items in priority order."""
```

```
    print(self.queue)
```

```
pq = PriorityQueue()
```

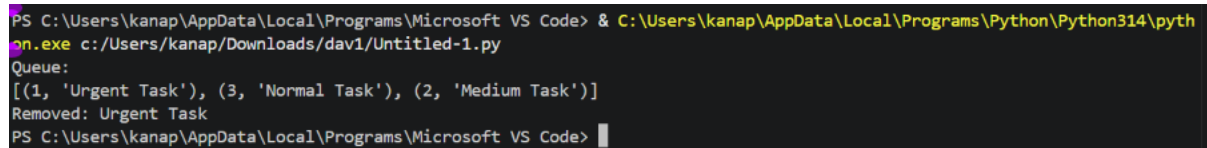
```
pq.enqueue("Normal Task", 3)
```

```
pq.enqueue("Urgent Task", 1)
pq.enqueue("Medium Task", 2)

print("Queue:")
pq.display()

print("Removed:", pq.dequeue())
```

OUTPUT:



```
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kanap\AppData\Local\Programs\Python\Python314\python.exe c:/Users/kanap/Downloads/dav1/Untitled-1.py
Queue:
[(1, 'Urgent Task'), (3, 'Normal Task'), (2, 'Medium Task')]
Removed: Urgent Task
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> █
```

Task Description #6 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

INPUT:

```
from collections import deque
```

```
class DequeDS:
```

```
    """Double-ended queue implementation."""
```

```
    def __init__(self):
```

```
        self.items = deque()
```

```
    def insert_front(self, item):
```

```
        """Insert an item at the front."""
```

```
        self.items.appendleft(item)
```

```
    def insert_rear(self, item):
```

```
        """Insert an item at the rear."""
```

```
        self.items.append(item)
```

```
    def remove_front(self):
```

```
        """Remove an item from the front."""
```

```
        if not self.items:
```

```
        return "Deque is empty"
    return self.items.popleft()
```

```
def remove_rear(self):
    """Remove an item from the rear."""
    if not self.items:
        return "Deque is empty"
    return self.items.pop()
```

```
def display(self):
    """Display the deque."""
    print(list(self.items))
```

```
d = DequeDS()
```

```
d.insert_front(20)
```

```
d.insert_front(10)
```

```
d.insert_rear(30)
```

```
d.display()
```

```
print("Removed Front:", d.remove_front())
```

```
print("Removed Rear:", d.remove_rear())
```

```
d.display()
```

OUTPUT:

```
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kanap\AppData\Local\Programs\Python\Python314\python.exe c:/Users/kanap/Downloads/dav1/Untitled-1.py
[10, 20, 30]
Removed Front: 10
Removed Rear: 30
[20]
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> |
```

Task Description #7 Real-Time Application Challenge –
Choose the Right Data Structure

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

INPUT:

```
class CafeteriaQueue:
```

```
    """Manage cafeteria orders using FIFO queue."""
```

```
def __init__(self):
    self.orders = []

def add_order(self, order):
    """Add a new order to the queue."""
    self.orders.append(order)

def serve_order(self):
    """Serve the first order in the queue."""
    if not self.orders:
        return "No orders available."

    return self.orders.pop(0)

def display_orders(self):
    """Display all pending orders."""
    if not self.orders:
        print("No pending orders.")
```

else:

print("\nPending Orders:")

for order in self.orders:

print("-", order)

cafeteria = CafeteriaQueue()

while True:

print("\n--- Cafeteria Order System ---")

print("1. Add Order")

print("2. Serve Order")

print("3. Display Orders")

print("4. Exit")

choice = input("Enter your choice: ")

if choice == "1":

```
order = input("Enter food order: ")  
cafeteria.add_order(order)  
print("Order added successfully.")
```

```
elif choice == "2":  
    served = cafeteria.serve_order()  
    print("Serving:", served)
```

```
elif choice == "3":  
    cafeteria.display_orders()
```

```
elif choice == "4":  
    print("Thank you!")  
    break
```

```
else:  
    print("Invalid choice.")
```

OUTPUT:

```
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kanap\AppData\Local\Programs\Python\Python314\python.exe c:/Users/kanap/Downloads/dav1/Untitled-1.py

--- Cafeteria Order System ---
1. Add Order
2. Serve Order
3. Display Orders
4. Exit
Enter your choice: 2
Serving: No orders available.

--- Cafeteria Order System ---
1. Add Order
2. Serve Order
3. Display Orders
4. Exit
Enter your choice: 1
Enter food order: PIZZA
Order added successfully.

--- Cafeteria Order System ---
1. Add Order
2. Serve Order
3. Display Orders
4. Exit
Enter your choice: 4
Thank you!
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code>
```

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue

- o Linked List
- o Binary Search Tree (BST)
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

INPUT:

```
class OrderQueue:
```

```
    """Manage e-commerce orders using FIFO queue."""
```

```
    def __init__(self):
```

```
        self.orders = []
```

```
    def add_order(self, order):
```

```
        """Add an order to the queue."""
```

```
        self.orders.append(order)
```

```
def process_order(self):
    """Process the oldest order."""
    if not self.orders:
        return "No orders available."

    return self.orders.pop(0)

def display_orders(self):
    """Display all pending orders."""
    if not self.orders:
        print("No pending orders.")
    else:
        print("\nPending Orders:")
        for order in self.orders:
            print("-", order)
```

```
system = OrderQueue()
```

while True:

print("\n--- Smart E-Commerce Order System ---")

print("1. Add Order")

print("2. Process Order")

print("3. Display Orders")

print("4. Exit")

choice = input("Enter your choice: ")

if choice == "1":

order = input("Enter product/order name: ")

system.add_order(order)

print("Order added successfully.")

elif choice == "2":

order = system.process_order()

print("Processing:", order)

```
elif choice == "3":
```

```
    system.display_orders()
```

```
elif choice == "4":
```

```
    print("Exiting system...")
```

```
    break
```

```
else:
```

```
    print("Invalid choice.")
```

OUTPUT:

```
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kanap\AppData\Local\Programs\Python\Python314\python.exe c:/Users/kanap/Downloads/dav1/Untitled-1.py

--- Smart E-Commerce Order System ---
1. Add Order
2. Process Order
3. Display Orders
4. Exit
Enter your choice: 1
Enter product/order name: 5
Order added successfully.

--- Smart E-Commerce Order System ---
1. Add Order
2. Process Order
3. Display Orders
4. Exit
Enter your choice: 3

Pending Orders:
- 5

--- Smart E-Commerce Order System ---
1. Add Order
2. Process Order
3. Display Orders
4. Exit
Enter your choice: 4
Exiting system...
PS C:\Users\kanap\AppData\Local\Programs\Microsoft VS Code> 
```